CRC Package Interface Specification

Current API design

Table of contents

1 Overview 1

2 Main arguments 2

3 Current workflow 2

4 Parser responsibilities 3
- 4.1 parse_captures() . . . 3
- 4.2 parse_outcome() . . . 3
- 4.3 parse_capture_formula() . . . 3
- 4.4 parse_outcome_formula() . . . 4
- 4.5 parse_misclass() . . . 4

5 Returned object 4

6 Next implementation steps 5

1 Overview

The package is centred around a single high-level function:

```
crc_fit(
  data,
  captures,
  capture_formula,
  outcome = NULL,
  outcome_formula = NULL,
  outcome_dist = c("ztnegbin", "ztpois"),
```

```
  misclass = NULL,
  latent_classes = 1L,
  control = NULL
)
```

The current implementation focuses on **validation and parsing**. Model fitting (EM algorithm) will be implemented later.

2 Main arguments

Argument	Description
`data`	One row per observed unit.
`captures`	One-sided formula defining capture indicators.
`capture_formula`	Log-linear capture model. `.latent` denotes the latent class.
`outcome`	`NULL`, a single outcome variable, or `c(lower=..., upper=...)`.
`outcome_formula`	Mean model for the outcome distribution.
`outcome_dist`	Currently `"ztnegbin"` or `"ztpois"`.
`misclass`	Misclassification specification for categorical variables.
`latent_classes`	Number of latent classes.
`control`	Numerical control options (reserved for future use).

3 Current workflow

```
crc_fit()

   match.arg(outcome_dist)
   validate_crc_input()
   parse_captures()
   parse_outcome()
   parse_misclass()
   parse_capture_formula()
   parse_outcome_formula()
```

At this stage the function returns an **unfitted model object** containing parsed specifications.

4 Parser responsibilities

4.1 `parse_captures()`

- validates the capture formula;
- verifies binary capture indicators;
- checks missing values;
- checks that every unit is observed by at least one source;
- returns a `crc_captures` object.

4.2 `parse_outcome()`

Supports:

- `NULL`;
- one exact outcome column;
- interval/right-censored outcomes via `lower`/`upper`.

Returns a standardized table with:

- `lower`
- `upper`
- `status`

4.3 `parse_capture_formula()`

Validates:

- one-sided formula;
- intercept;
- untransformed variables only;
- presence of all capture main effects;
- hierarchical interactions;
- `.latent`;
- allowed interactions:
  - capture × capture,
  - capture × `.latent`.

Returns a `crc_capture_formula` object.

4.4 `parse_outcome_formula()`

Validates:

- one-sided formula;
- intercept;
- categorical predictors;
- hierarchical interactions;
- optional use of `.latent`.

Returns a `crc_outcome_formula` object.

4.5 `parse_misclass()`

Expected structure:

```
misclass = list(
  occupation = list(
    matrix = Pi_occ,
    true_if = ~ source_1
  ),
  contract = list(
    matrix = Pi_contract
  )
)
```

Responsibilities:

- validates confusion matrices;
- validates category levels;
- validates `true_if`;
- constructs logical vectors identifying observations with known true category;
- returns a `crc_misclass` object.

5 Returned object

Currently `crc_fit()` returns an object of classes

```
c("crcfit_unfitted", "crcfit")
```

containing:

- parsed capture specification,
- parsed outcome specification,
- parsed capture model,
- parsed outcome model,
- parsed misclassification specification,
- model metadata.

6 Next implementation steps

1. Build internal model matrices.
2. Initialize parameters.
3. Implement the EM algorithm.
4. Compute standard errors.
5. Add `summary()`, `print()`, and `predict()` methods.

CRC Package Interface Specification

Model-matrix implementation addendum

1 Model-matrix infrastructure

The package now contains initial infrastructure for converting parsed model specifications into complete cell grids and observation-level latent states. The implementation remains separate from `crc_fit()` until the remaining matrices and their alignment rules are defined.

1.1 Capture-model matrix

`build_capture_matrix()` constructs the Cartesian product of all capture histories, categorical levels used by the capture model, and latent classes. For misclassified variables, true-category levels and their ordering are obtained from the corresponding confusion matrices. The function returns:

- `cells`, the complete capture-model cell grid;
- `matrix`, the design matrix generated from the parsed terms;
- `unobserved`, an indicator of all-zero capture histories.

With J capture sources, K latent classes, and categorical variables with $C_1, \ldots, C_p$ levels, the grid contains $2^J K \prod_{q=1}^{p} C_q$ rows.

1.2 Initial observation states

`build_observation_states()` repeats every observed unit once for each latent class. It retains the variables needed by the capture model, outcome model, and misclassification mechanisms. It also keeps aligned copies of the standardized outcome and observed values of misclassified variables. An observation identifier maps every expanded row back to its original unit.

1.3 Misclassified-variable expansion

`expand_misclass_states()` expands the current states over every possible true level of one misclassified variable. Each expanded row receives the contribution

$$\Pr(\widetilde{G}_j = h \mid G_j = g)$$

from its confusion matrix. When `true_if` applies, the observed category receives probability one and all other candidate categories receive probability zero.

For several misclassified variables, separate confusion matrices are used and their contributions are multiplied. This corresponds to conditional independence of the misclassification mechanisms given their respective true categories. Dependent mechanisms may be supported in a future extension. These contributions are likelihood terms, not posterior probabilities, and must not be normalized at this stage.

2 Required next steps

The model-matrix work should continue in the following order.

1. Add a small wrapper that calls `expand_misclass_states()` for every variable in `misclass`. When no mechanism is specified, it should assign a contribution of one to every state.
2. Build the outcome-model design matrix from the fully expanded state data. It must remain row-aligned with the standardized outcome, observation identifiers, and misclassification contributions.
3. Define indices linking each observation state to the corresponding row of the complete capture-cell matrix. This avoids repeatedly matching capture histories, true categorical values, and latent classes during the EM iterations.
4. Create a single internal model-matrix object containing the capture grid, capture design matrix, expanded observation states, outcome design matrix, alignment indices, and outcome metadata.
5. Integrate construction of this object into `crc_fit()` while retaining the current unfitted-object behavior.
6. Add focused tests for dimensions, row alignment, factor-level order, all-zero capture cells, multiple misclassified variables, and `true_if` restrictions.

Only after these structures and invariants are established should development proceed to parameter initialization and the EM algorithm. In the E-step, the misclassification contributions will be combined with capture-model expected cell frequencies and outcome likelihood contributions, and then normalized within each observed unit to obtain posterior state probabilities.